CS 5326: Programming Assignment 3 Technical Report

Student Name: Emaan Roshan

Roll Number: 25030001

Date: 7 November 2026

Section 1: Benchmark Results and Compute Scaling

1.1 Summary Results Table

I ran all six configurations on the 50-problem benchmark. SC-5 and the three PRM runs use the same five sampled paths per problem (indices 0–4), SC-15 adds ten more (5–14), and the three PRM rows reuse one set of judge verdicts. So the PRM rows differ only in how the step scores are combined. Accuracy is exact match with the strict `Final Answer:` parser from the manual, and a missing or unparseable answer counts as wrong.

Method	Acc. (%)	95% bootstrap CI	Gen calls	Judge calls	Total calls	Gen tok / prob.	Judge tok / prob.	Time (s)
Greedy (N=1)	62.0	[48.0, 74.0]	50	0	50	193.6	0	334
Self-Consistency (N=5)	66.0	[54.0, 78.0]	250	0	250	1334.2	0	479
Compute-Matched SC (N=15)	**74.0**	[62.0, 86.0]	750	0	750	4313.8	0	1109
PRM Best-of-5 (min)	70.0	[57.95, 82.0]	250	1079	1329	1334.2	565.8	2573
PRM Best-of-5 (product)	72.0	[60.0, 84.0]	250	1079	1329	1334.2	565.8	2573
PRM Best-of-5 (step penalty, λ=2)	70.0	[56.0, 82.0]	250	1079	1329	1334.2	565.8	2573

Table 1. Results. Intervals are 95% bootstrap percentile intervals (1,000 resamples over problems, seed 5326). The average path has $\bar{K}$ = 1079/250 = 4.32 steps, which is about 26.6 LLM calls per problem for PRM search.

1.2 Scaling Plots

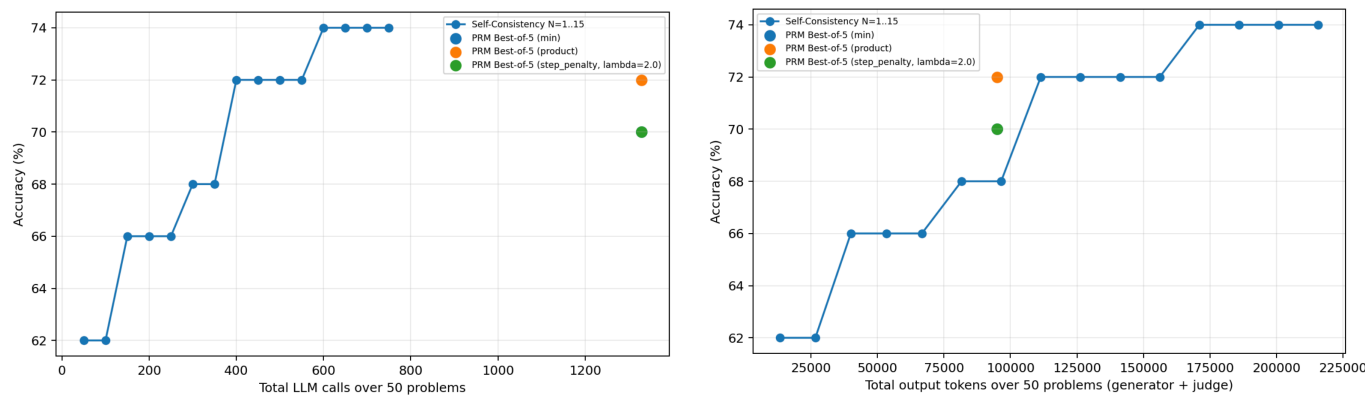


Figure 1. Accuracy vs. total LLM calls (left) and vs. total output tokens, generator plus judge (right). Blue line is self-consistency for N = 1 to 15. PRM min and PRM step penalty have the same cost and the same accuracy (70%), so the green point sits on top of the blue PRM-min point.

1.3 Statistical Analysis and Discussion

Paired win/loss counts for all PRM variants against both SC baselines are in Table 2. For the two comparisons the template asks about, PRM-BoN (min) vs SC-5 is 3 wins, 1 loss and 46 agreed (32 both correct, 14 both wrong). PRM-BoN (min) vs SC-15 is 2 wins, 4 losses and 44 agreed (33 + 11).

PRM method	Baseline	PRM wins	PRM losses	Both correct	Both wrong	Δ acc. (pp)	95% CI of Δ (pp)	Exact McNemar p
min	SC-5	3	1	32	14	+4.0	[−4.0, 12.0]	0.63
min	SC-15	2	4	33	11	−4.0	[−14.0, 6.0]	0.69
product	SC-5	4	1	32	13	+6.0	[−2.0, 14.05]	0.38
product	SC-15	2	3	34	11	−2.0	[−10.0, 6.0]	1.00
step penalty (λ=2)	SC-5	3	1	32	14	+4.0	[−4.0, 12.0]	0.63
step penalty (λ=2)	SC-15	2	4	33	11	−4.0	[−14.0, 6.0]	0.69

Table 2. Paired win/loss counts on the same 50 problems, with a paired bootstrap CI for the difference and an exact two-sided McNemar p-value.

No. With 50 problems the standard error of an accuracy around 70% is about 6.5 pp, so all the intervals in Table 1 are roughly ±12 pp wide and overlap a lot. The paired comparison is sharper but still has little to work with: a PRM method and an SC baseline disagree on only 4 to 6 problems out of 50, so a 4 pp gap is just two problems. Every paired CI in Table 2 contains zero and the smallest p-value is 0.38. Even SC-15 against greedy (+12 pp, 8 wins and 2 losses) only reaches p = 0.11.

What I can say is that every multi-sample method beats greedy by 4 to 12 pp, which fits the idea that extra test-time compute helps. PRM Best-of-5 is ahead of SC-5 by 4 to 6 pp and behind SC-15 by 2 to 4 pp, but neither gap is bigger than noise. The PRM did change answers: it differs from SC-5 on 8 problems (min), 13 (product) and 11 (penalty). In `test_16` the five SC-5 samples all disagreed, so SC-5 took the earliest (wrong) one. The PRM vetoed three paths and the earliest of the two fully valid survivors (125 vs 221) happened to be the right one. In `test_46` a 3-vs-2 majority for 44 was overruled because all three of those paths had flagged steps. `test_40` looks like `test_16`. The one loss, `test_14`, goes the other way: a wrong path (36) was judged fully valid, two of the three paths giving the correct plurality answer 18 were vetoed, and the 36-vs-18 tie went to the earlier index.

One more caveat, covered in Section 3: 134 of 800 completions come back as `None` under the strict parser, and a more lenient parser changes how PRM compares with SC-15.

Section 2: Path Score Aggregation Ablation

2.1 Aggregation Ablation Table

Strategy	λ	Acc. (%)	Tie rate (%)	Mean tied-set size	Problems with a unique top path
min	—	70.0	88.0	3.26	6
Product	—	72.0	56.0	2.04	22
Step penalty	0.0	70.0	82.0	3.04	9
Step penalty	0.5	70.0	84.0	3.02	8
Step penalty	2.0	70.0	84.0	3.02	8

Table 3. Ablation from `aggregation_ablation.json`. Tie rate is the share of problems where more than one candidate has the top score. The last two columns I computed from `per_problem.jsonl`.

2.2 Analytical Breakdown

Why product behaves like min, and where it differs.

Take a path with T steps, k of them judged INVALID. With binary verdicts the three rules become simple functions of k and T. Each step probability is clamped to 0.99 (VALID) or 0.01 (INVALID), so the product is

$$S_{\text{product}} = 0.99^{T-k} \times 0.01^{k}$$

Min is +1 if k = 0 and −1 otherwise, and the penalty rule is $1 - 2k/T - \lambda k$.

Product is strictly decreasing in k, because every invalid step multiplies the score by 1/99. So any path with an invalid step ranks below every fully valid path, which is exactly what min does. The only real difference is among equally clean paths: product has the factor 0.99^{T}, which slightly prefers shorter paths, while min and penalty give every fully valid path exactly 1.0. That is why product's tie rate is 56% and min's is 88%, and why product has a unique top path in 22 problems against 6 for min.

For the penalty rule, once λ is 0.5 or more a single invalid step costs at least 0.5 + 2/T, so the ranking is dominated by k. λ = 0.5 and λ = 2.0 picked the same answer on every problem. A fully valid path scores 1 for any λ, so λ can only matter when no candidate is fully valid, which happens in 6 of the 50 problems. With λ = 0 the score is 1 − 2k/T, the fraction of invalid steps, and the tie rate drops a little (82%) with no change in accuracy.

Accuracy is 70% for min and for all three penalty settings. The extra 2 pp for product is one problem, `test_06`. There, four fully valid paths gave 50, 38, 53 and 64, so min voting hit a four-way tie and took the earliest (50), while product took the shortest path (64). That could be a real length effect, but one problem is not enough to say so.

Filter-then-vote.

Because paths are filtered by score first and the vote comes second, min-pooled PRM is basically self-consistency over the paths the judge did not veto. At least one path scores +1 in 44 of 50 problems, and in 18 of them all five paths are fully valid, so PRM-min is identical to SC-5 there. Overall 44 problems have a tied top set. In 23 of them the tied paths already agree on one answer, and in the other 21 they disagree, so the vote decides. If ties were broken by taking the first candidate instead, the PRM would just return one random sample from the survivors. The PRM only matters in the 8 problems where its answer differs from SC-5 (3 wins, 1 loss, 4 both wrong for min). It can hurt when the veto removes paths behind the correct plurality, as in `test_14`.

Optional bonus. I did not implement the `product_logprob` variant, so I make no claim about it.

Section 3: Failure Mode and Judge Error Analysis

3.1 Qualitative Failure Cases

Case 1: Judge false positive (flawed step marked VALID). This is a false negative in the INVALID-positive matrix of 3.2. None of my 30 audited rows is of this kind (the zero cell), so this is the closest example I found in the full logs.

- **Problem ID / Question:** `gsm8k_test_20`. Marissa hikes a 12-mile trail. She walks the first 4 miles in 1 hour and the next 2 miles in another hour. What speed does she need for the rest to average 4 mph? (Answer: 6.)
- **Flawed step:** candidate 0, step 3: “Required speed: 6 miles / (2 hours) = 3 miles per hour.”
- **Judge verdict and rationale:** VALID. “The calculation correctly applies the speed formula using the remaining distance and total time.” All three steps got +1, and candidate 1, which made a different conceptual mistake (6 mi / 1.5 h = 4 mph), also passed.
- **Root cause:** the division is right but the 2 hours is time already spent. The divisor should be the remaining time, 12/4 − 2 = 1 hour. The judge seems to check the arithmetic of what is written, not whether the setup answers the question, so wrong formulas and misused quantities get through. `test_06` (all 15 remaining glasses priced at $3) and `test_09` (distances added instead of subtracted on the way home) look the same.

Case 2: Judge false negative (valid step marked INVALID). This is a false positive in the matrix of 3.2.

- **Problem ID / Question:** `gsm8k_test_03`. Josh buys a house for $80,000, spends $50,000 on repairs, and the value rises by 150%. What is his profit? (Answer: 70,000.)
- **Valid step flagged as invalid:** candidate 0, step 1: “Increase in value = 150% of purchase price = 1.5 × $80,000 = $120,000.”
- **Judge verdict and rationale:** INVALID. “The increase in value should be calculated based on the total cost, not just the purchase price.” On other candidates the same judge flagged the opposite choice (“Total cost × 150% … should be 150% of the original cost before repairs”).
- **Root cause:** the rationales contradict each other, so they are not a real check. Of the 8 false alarms in my audit, 5 are on `test_05` and say the step “should instead” compute something else, so the judge is comparing against its own preferred plan and not checking local validity. One is an empty planning step, and 2 are this Josh misreading. Under min-pooling a single false flag removes an otherwise correct path.

Case 3: Mode collapse / shared flawed premise.

- **Problem ID / Question:** `gsm8k_test_13`. Carlos plants a lemon tree for $90. It gives 7 lemons a year at $1.5 each and costs $3 a year to maintain. How many years until he starts earning money? (Answer: 13.)
- **Common flaw across candidates:** net profit is $7.50 a year and 90 / 7.5 = 12 exactly, so Carlos only breaks even after 12 years and starts earning in year 13. None of the 15 samples answered 13. Eight said 12 and the rest were arithmetic slips (11, 3, 1, 21)

or format failures. Every 12-path got +1 on every step, including candidate 3, which writes that it needs net profit “to be greater than or equal to zero”.

- **Root cause:** the judge only vetoed the sloppy path (candidate 4, answer 11). Filtering removed noise but not the shared off-by-one, and voting then pushed 12 through: SC-5, SC-15 and all three PRM rules output 12. The judge and generator are from the same model family, so they probably share blind spots. Six problems (12%) have no correct path in 15 samples (`test_08, 09, 13, 15, 44, 47`), which limits any selection method. Pass@1 is 62%, pass@5 is 74% and pass@15 is 88%.

3.2 Human Audit of Judge Decisions

I took the first 15 judge-INVALID steps and the first 15 judge-VALID steps in the audit file (190 INVALID steps exist in total) and labelled each one by local validity, meaning I assumed the earlier steps were correct. INVALID is the positive class.

Metric	Count / Value
True Positives (Judge: INVALID, Human: INVALID)	7
False Positives (Judge: INVALID, Human: VALID)	8
True Negatives (Judge: VALID, Human: VALID)	15
False Negatives (Judge: VALID, Human: INVALID)	0
Judge Precision (INVALID detection)	**46.7%** (7/15)
Judge Recall (INVALID detection)	**100%** (7/7)

Table 4. Audit results for 30 hand-labelled steps (15 judge-INVALID, 15 judge-VALID). Accuracy is 22/30.

Because the matrix has a zero cell, here is why. About half of the judge's INVALID flags are false alarms (8 of 15). The perfect recall should not be trusted, because the sample is not random. All 15 VALID rows come from `test_01`, an easy problem where every path was correct, so there was nothing for the judge to miss. The INVALID rows come from just two problems (`test_03` and `test_05`). Looking at all the data instead: of 127 wrong paths among the five shared samples, 52 (41%) had every step marked VALID, and of the 87 wrong paths with a parseable answer, 59 (68%) had at least one flagged step. Meanwhile 16 of 123 correct paths (13%) were flagged. So the judge both rejects good steps and lets bad ones through.

Format failures (extra analysis)

Under the strict parser 129 of 750 sampled candidates (17.2%) and 5 of 50 greedy outputs (10%) give `None`. Only 19 of those 134 have no `Final Answer` line at all (14 hit the 512-token limit). The other 115 have the line but the parser rejects it, for example “Final Answer: 60%”, “Josh made a profit of $70,000.”, “Kylar needs to pay $64 for the glasses” or “\$460”. In `test_15` all 15 samples and the greedy output say the correct 60% but score `None`, so every method fails that problem. To see how much this matters I re-parsed the saved completions with a simple lenient rule (take the number in the final-answer line) and re-ran the selection rules on the saved step scores. My re-implementation reproduces all six official accuracies under the strict parser. This is a sensitivity check only, not the official result.

Parser	Greedy	SC-5	SC-15	PRM min	PRM product	PRM penalty
Strict (official)	62	66	74	70	72	70
Lenient (sensitivity)	66	70	76	76	74	76

Every method improves, PRM a bit more (+2 to +6 pp, against +2 to +4 pp for the others), since the judge scores the reasoning whether or not the last line parses. With lenient parsing PRM-min and PRM-penalty tie SC-15, so the PRM-vs-SC-15 comparison depends on how strict the parser is.

3.3 Discussion Question

The bigger judge is good at things that can be checked locally: arithmetic slips and inconsistencies. It flagged 68% of the wrong parseable paths, and in `test_13` it caught the path that mixed the planting cost into the yearly profit. It is weak on conceptual errors, boundary conditions and misread questions (Cases 1 and 3), and it sometimes rejects valid steps that are phrased unusually (Case 2). Using a bigger model also means the judge does not copy the generator's slips one for one. If the 1.5B model verified itself, I would expect more of the same conceptual errors to pass, because the model that wrote the step would be checking it with the same understanding, and it would likely go easy on its own wording and answers. Its arithmetic checking would also be weaker, and

unparseable verdicts (which default to INVALID under the manual's rule) would add false alarms. The filter would be close to useless and PRM search would drift towards plain self-consistency.

Section 4: Test-Time Compute Efficiency Analysis

Defining 10× greedy compute

Greedy generated 9,682 tokens in total (193.64 × 50), so 10× greedy is a budget of **96,820 output tokens**, counting generator and judge tokens from token IDs and excluding prompts.

Method	Output tokens	Within 10× budget?	Acc. (%)
SC N=5	66,710	yes	66
SC N=7 (largest SC in budget)	96,506	yes	68
PRM Best-of-5: min / product / step penalty (66,710 gen + 28,291 judge)	95,001	yes	70 / 72 / 70
SC N=8	111,404	no (+15%)	72
SC N=12	170,996	no (+77%)	74
SC N=15	215,690	no (2.2×)	74

Table 5. Selected rows of `compute_budget.csv`.

Generator and judge cost

The judge makes 1,079 of the 1,329 calls (81%) but produces only 28,291 of the 95,001 output tokens (30%). A generator call emits about 267 tokens and a judge call about 26. So output tokens understate what the verifier costs. Each judge call reads the question and the whole prefix, and that prefill is not counted. Wall-clock time shows it: PRM took 2,573 s against 479 s for SC-5 (5.4×) and 1,109 s for SC-15 (2.3×), and about 2,095 s of the PRM time (81%) was judging. SC-15 is also not really “compute-matched”: it uses 56% of the PRM call count (750 vs 1,329) but 2.3× its output tokens. Matched by tokens, the fair baseline is SC-7 (96,506 tokens).

Strategic recommendation

Inside the budget, PRM scores a little higher (70 to 72% against 68% for SC-7), but that is one or two problems (PRM-product vs SC-7: 3 wins, 1 loss, p = 0.63; PRM-min vs SC-7: 2 wins, 1 loss). The scaling curve shows why the gap is small. SC goes up in steps, +4 pp at N = 3, +2 at N = 6, +4 at N = 8 and +2 at N = 12, then stays at 74%. PRM at 70 to 72% is where SC lands at N = 8, which costs only 17% more tokens than the PRM (and SC-8 vs PRM-product is 2 wins, 2 losses). I would put the budget into sampling with a larger N. Accuracy is about the same, it is 2 to 5 times faster (SC-15: 2.3×, SC-5: 5.4×), and it does not need a second 7B model in memory. The catch is the parser result in Section 3: with lenient parsing PRM-min and PRM-penalty tie SC-15 at 76%, so this recommendation rests mostly on cost and not on accuracy. PRM would make more sense with cheaper verification (shorter prompts, cached prefixes, batched judging), with a judge that is trained or calibrated, or when the limit is specifically output tokens and a verifier is already loaded.

Section 5: Limitations and Reproducibility

Hardware and runtime environment

- **Hardware and models.** Experiments were run on Kaggle with two NVIDIA Tesla T4 GPUs. The Qwen2.5-1.5B-Instruct generator ran in FP16 on GPU 0, and the Qwen2.5-7B-Instruct judge ran with 4-bit NF4 quantization (double quantization, FP16 compute) on GPU 1.
- **Software environment.** Python 3.13.15, PyTorch 2.11.0+cu128, Transformers 5.16.1, Accelerate 1.14.0 and BitsAndBytes 0.50.2. The analysis scripts used NumPy 2.1.3 and Matplotlib 3.10.0.
- **VRAM.** The manual's estimate is about 3.1 GB for the FP16 generator and about 5.5 GB for the NF4 judge, one model per T4. I queried `torch.cuda.memory_allocated` and `max_memory_allocated` after the run in a fresh process, and both returned 0.00 GB on both GPUs, so I have no measured peak and use the nominal figures.
- **Runtime.** The recorded per-problem execution times total about 3,538 s (59 minutes): greedy 334 s, SC-5 479 s, the 10 extra samples 630 s and judging 2,095 s. This excludes model loading and initialization. Total end-to-end Kaggle wall-clock time was not separately recorded.

- **Seeds and decoding.** `transformers.set_seed(5326 + problem_index)` before each problem (saved as `seed`). Greedy: `do_sample=False` with temperature, top-p and top-k set to None. Sampling: temperature 0.7, top-p 1.0, top-k 0, repetition penalty 1.0. Max new tokens 512 for the generator and 96 for the judge. The SC curve uses the first N shared samples, so N = 5 and N = 15 match the SC rows.

Methodological limitations

- **Limits of the evaluation.** One run, 50 problems, intervals of about ±12 pp and only 4 to 6 disagreements per comparison. Sampled completions are 38% longer than greedy ones (266.8 vs 193.6 tokens) and 70 of 750 hit the 512-token cap. The audit covers 30 steps from 3 problems, picked in file order and labelled by one person, so its recall number means little.
- **Prompted judge vs a trained PRM.** The 7B judge is a general instruction-tuned model asked for a verdict. It was not trained on human step labels the way PRM800K-style models are (Lightman et al., 2023). Its verdicts are binary and uncalibrated, so the "product" score is not a real joint probability. Its explanations are inconsistent (Case 2) and its mistakes are correlated with the generator's (Case 3). It also cannot act as a value function, because it scores finished steps and not how promising a partial solution is, so it only supports Best-of-N reranking and not beam or tree search. A trained PRM would have seen real solution errors and could give calibrated graded scores. The 4-bit quantisation may add some noise too, and the strict parser and the "unparseable verdict means INVALID" rule from the manual both bias the results, as shown above.